# 0230. Kth Smallest Element in a BST

um | Category: Trees | LeetCode: https://leetcode.com/problems/kth-smallest-ele

# Problem

- Return the k-th smallest value (1-indexed) in a BST.

# Intuition

- Inorder traversal of a BST visits values in sorted order, so the k-th visited node is the answer.
- An iterative stack lets us stop as soon as we reach k.

# Brute Force

- Collect all values, sort, index: O(n log n).

# Optimal Approach

- Cur = root, stack empty.
- Push all left children; pop a node; k--; if k == 0 return it.
- Move to its right child and repeat.

# Worked Example

```
[5,3,6,2,4,null,null,1], k = 3.

push 5,3,2,1
pop 1 (k=2) ; pop 2 (k=1) ; pop 3 (k=0) -> answer 3
```

# Java Solution

```java
public class KthSmallestElementInBST {
    public int kthSmallest(TreeNode root, int k) {
        Deque<TreeNode> st = new ArrayDeque<>();
        TreeNode cur = root;
        while (cur != null || !st.isEmpty()) {
            while (cur != null) { st.push(cur); cur = cur.left; }
            cur = st.pop();
            if (--k == 0) return cur.val;
            cur = cur.right;
        }
        throw new IllegalArgumentException("k larger than tree size");
    }
}
```

# Complexity

- Time: O(h + k). Space: O(h).

# Pitfalls

- Follow-up (frequent inserts/deletes): augment nodes with subtree sizes for O(h) queries.